



Quick Quote Engine

The problem

Merchants shipping a parcel need to choose a courier in seconds. Fincart quotes three carriers and shows the merchant a single comparable list.

The three carriers do not resemble each other. Two are external APIs we integrate against. One is a carrier we hold a contracted rate card with, so we compute those prices ourselves. All three are slow, unreliable, or both.

Your job is the quoting service and a thin UI on top of it.

What we give you

The `starter/` directory contains material you should treat as pre-existing:

Path	What it is
<code>starter/providers/swiftpost.ts</code>	Vendor SDK for SwiftPost. Do not modify.
<code>starter/providers/atlas.ts</code>	Vendor SDK for Atlas Logistics. Do not modify.
<code>starter/rate-cards/meridian.json</code>	Our contracted rate card with Meridian Freight. Data only the pricing logic is yours to write.
<code>starter/lib/rng.ts</code>	Support code the vendor SDKs depend on. Do not modify.
<code>.env.example</code>	Fixed FX rates, the provider seed, and other configuration. Copy to <code>.env</code> .

The two vendor SDKs are stand-ins for third-party integrations. You have no control over their latency, their failure rate, or the shape of what they return. Read their header comments the way you would read vendor API docs, they are accurate and complete.

Copy these files into your repo wherever your structure wants them. The `import` line at the top of each provider is the **only** line you may change in the three read-only files, and only to fix the path to `rng.ts`.

`starter/` as shipped type-checks under `tsc --strict` with `@types/node` installed. If it does not in your repo, the path is wrong.

Part 1 Quote aggregation (backend)

1.1 The endpoint

One **authenticated** endpoint that takes a shipment and returns rates from all three carriers.

A shipment is:

```
{
  origin: { countryCode: string; postcode: string; timezone: string },
  destination: { countryCode: string; postcode: string },
  parcel: { weightKg: number; lengthCm: number;
            widthCm: number; heightCm: number },
  quotedAt?: string // ISO 8601. Defaults to now. See 1.2.
}
```

`origin.timezone` is an IANA zone name (`America/New_York`), not an offset.

The shipment body carries no merchant identity. The calling merchant's **account tier** and **carrier account reference** come from the JWT and the merchant record, not from the request body. Both are needed to price a quote: see 1.2 and the Atlas vendor docs.

Every rate you return to the client must carry, at minimum: carrier, service, **base price**, **tax**, **total**, and an estimated delivery date or range.

Money and units

All monetary amounts presented to the merchant are in **USD minor units** (integer cents). Where a conversion is required, use the fixed rate in `FX_RATES` (provided in `.env.example`), no live FX lookups.

Amounts must be exact. We will check arithmetic against reference cases, so the following are pinned rather than left to you:

- **Rounding:** round half away from zero (half-up). `745.5` becomes `746`. Do not use banker's rounding. Round only at the points a rule tells you to, and carry full precision in between.
- **FX:** convert each monetary component separately, in minor units, then round. Do not convert a total and derive the components from it.
- **Mass:** 1 kg = 2.20462262 lb.
- **Length:** 1 in = 2.54 cm exactly.
- Pass unrounded converted values to vendors that want other units. Do not pre-round a request-side conversion.

1.2 The Meridian rate card

Implement Meridian pricing from `meridian.json`. The `$semantics` block in that file is the specification; follow it precisely. It covers rounding, chargeable weight, weight breaks, fuel, oversize, remote-area, handling, business-day transit and tax.

Two of those rules will not fit neatly alongside the other two carriers, because they need information a pure (shipment) => Rate function does not have. That is deliberate and it is the interesting part of this exercise. How you accommodate them is up to you.

quotedAt exists so that a quote is reproducible. Anything time-dependent in your pricing must derive from it rather than from a direct clock read, that is also what makes your tests deterministic.

Meridian is a local calculation, so it has no network latency.

1.3 Latency contract

| This is a hard requirement, not an aspiration.

We will fire 200 sequential quote requests at your endpoint on the **US → GB** lane with PROVIDER_SEED fixed, and measure:

- **p95 time-to-first-rate < 800 ms** from request start to the first usable rate reaching the client.
- **p95 time-to-complete < 3,500 ms** all rates that are going to arrive have arrived, or been given up on.

Both numbers are measured **at the client**, not server-side. Include the script you used (npm run bench or similar) so we can reproduce your figures. Expect a run to take around ten minutes.

The vendor SDKs take up to 3 seconds each. Both numbers above are achievable on this lane; how is your call.

Two notes on the seed, because they will cost you an afternoon otherwise:

- PROVIDER_SEED is read **at module load**. If you configure dotenv after importing a provider, the seed is silently ignored and nothing reproduces.
- Each provider holds one RNG stream for the process, and different code paths draw a different number of values from it. Your own runs will reproduce exactly; a run with a different retry strategy will not.

1.4 Resilience

Carriers fail ~18–20% of the time and Atlas rate-limits. A partial result is a good result; a hung request is not. Make the failure and degradation behaviour of your system a deliberate design choice rather than a default, and be ready to explain the choice.

1.5 Validation

Zod schemas, shared between client and server via a packages/shared workspace. Reject zero or negative weight, impossible dimensions, malformed country codes, malformed timezone names, and malformed quotedAt.

Carrier serviceability is **not** a validation concern, a lane no carrier serves is a valid request with an empty result, and Part 3 asks you to render it as such. Do not encode carrier coverage in the shared schema.

Part 2 Auth and persistence

- Persist every quote request **and** its results.
- POST /auth/login issuing a JWT. Seeded merchants are fine; no signup, no password reset, no refresh tokens.
- GET /quotes returns **only the calling merchant's** history.
- Merchant accounts carry an account tier (standard | plus | enterprise) and a carrier account reference. The rate card uses the tier; Atlas needs the reference.
- Seed at least one standard and one enterprise merchant, and put both sets of credentials in the README. We test tier-dependent pricing.
- One line in your README on why you chose the database you chose.

Required test: an automated test proving merchant A cannot read merchant B's quote history. Not a comment saying you would, a test that fails if the scoping breaks.

Part 3 Frontend

Styling is explicitly not evaluated. Unstyled semantic HTML scores the same as a polished design system. No logos, no brand colours, no responsive grid, no animation. Do not spend time here.

What we do look at:

- A single-page form for the shipment shape above, using React Hook Form + the shared Zod schema.
- Rates rendered as they arrive, consistent with your latency contract. The fast carrier should appear before the slow ones.
- Four states handled honestly: loading, partial success (some carriers failed — say which), complete, and no-service-for-this-lane.
- A retry path when things fail.
- Keyboard-operable form with real <label> elements and errors associated to their inputs.

Part 4 Tests

Quality over coverage. We would rather see four tests that would catch a real regression than forty that assert mock behaviour.

- **Determinism is required.** Your tests must pass on repeated runs without flaking, despite the carriers' randomness and the time-of-day rules. Fixing `PROVIDER_SEED` before the providers load, and driving time from `quotedAt` rather than the system clock, is enough to get there.
- Rate card logic needs real cases, including the boundaries. Weight-break edges, the surcharge cap, and the rounding rule all have exact answers.
- Aggregation needs a partial-failure case and a slow-carrier case.
- One integration test hitting a real HTTP endpoint.
- The tenant-isolation test from Part 2.
- One frontend test of your choosing.

Part 5 README

Short. Three things:

1. **Run it.** `docker-compose` up or equivalent, plus seeded login credentials for both merchant tiers. If we cannot run it in under five minutes we cannot assess it.
2. **Your latency numbers**, and one paragraph on how you hit them.
3. **Shortcuts.** *Max 250 words.* Name the specific shortcut you are least comfortable with, why you took it, and what you would do with two more days. Concrete and specific. We are not looking for an essay on scaling.

No deployment required. Local `docker-compose` is the expectation.

Also include `PROMPTS.md` if you used AI assistance — we encourage it, we just want to see how. It is not scored; we read it for context.

Questions

Ambiguity in this document is fair game to ask about — email us. Ambiguity in `meridian.json` should be resolved by re-reading `$semantics`, which is authoritative.